

QUENTIN GUILLEREY

Customer Operations and Customer Success Manager

guillerey.finance@gmail.com | linkedin.com/in/quentin-guillerey | Columbia, MO

CSR AUTOMATION TOOLKIT

AI-Augmented Service Delivery Prototype | 2024–2025

Python | OpenAI API | Tkinter GUI | IMAP/SMTP | Slack Webhooks | Google Sheets

STATUS: Working prototype, not yet at production scale.

Architecture is fully functional end-to-end (classification, auto-reply, alerting, audit logging). Currently covers 5 response categories as a proof of concept. Scaling to the team's full ~1,300-entry query library is the active next phase. Code is public: github.com/Quentin-Guillerey/csr-automation-toolkit

THE PROBLEM

I was running U.S. operations for a Fortune 500 enterprise account handling 3,000+ customer contacts per month. The team had no standardized response system, no audit trail, and no mechanism to flag and route follow-up tasks. Agents spent most of their time on manual triage and copy-paste work, leaving little capacity for the interactions that actually moved the needle. No one was going to build the solution. So I started building one.

WHAT I BUILT

- **Tkinter GUI.** Desktop application built for non-technical operators. No command line required, no IT dependency.
- **Keyword classifier with AI fallback.** Rule-based keyword matching as the primary engine, with an OpenAI fallback for queries outside known patterns. Currently 5 core categories as a proof of concept; designed to extend to the team's full query taxonomy.
- **Auto-reply engine.** IMAP email reader with SMTP auto-response mapped to classifier output.
- **Slack alerting.** Every query triggers a notification. High-priority items are flagged [FOLLOW-UP NEEDED] for manual escalation (Asana/Zapier auto-routing is planned, not yet wired in).
- **Google Sheets audit log.** Full interaction history: timestamp, customer contact, query, response sent, follow-up flag. Built for compliance and performance analysis from day one.

NEXT STEPS

- Expand classifier coverage from 5 categories to the full operational query taxonomy
- Build a feedback loop so misclassified queries get flagged and folded back into the rule set
- Package as a standalone executable for non-technical team rollout (no Python install required)
- Wire up Asana/Zapier auto-routing for flagged follow-ups
- Measure real performance once deployed at scale: time saved, escalation rate, audit coverage

WHY THIS MATTERS

This project is not a side experiment. It is what I do when I see a gap in a business I am responsible for: diagnose the operational problem, scope a practical solution, and build a working proof of concept without waiting for budget approval or a specialist. I built the architecture, put it in front of real use, and I'm scaling it deliberately rather than overstating where it stands today.

The instinct behind it applies to any operations role: walk into a business, understand how its processes, data, and tools connect, identify where automation creates real leverage, and execute without over-engineering the solution.